12. SP1 CLI 도구 분석

개요

SP1 CLI(`cargo prove`)는 SP1 프로젝트 개발을 위한 명령줄 도구입니다. 이 도구는 프로젝트 생성, 프로그램 빌드, 툴체인 관리, 검증 키 생성 등의 기능을 제공하여 SP1 개발 워크플로우를 간소화합니다.

12.1 CLI 명령어 구조

기본 사용법

```
cargo prove <SUBCOMMAND>
```

사용 가능한 서브커맨드

| 명령어 | 설명 |

|--------|------|

| `new` | 새로운 SP1 프로젝트 생성 |

| `build` | SP1 프로그램 컴파일 |

| `build-toolchain` | SP1 툴체인 빌드 |

| `install-toolchain` | SP1 툴체인 설치 |

| `vkey` | 프로그램의 검증 키 해시 조회 |

12.2 프로젝트 생성 (`cargo prove new`)

기본 사용법

```
cargo prove new <PROJECT_NAME> [OPTIONS]
```

템플릿 옵션

Bare 템플릿

```
cargo prove new my-project --bare
```

- 기본적인 프로그램과 스크립트만 포함
- 최소한의 구조로 빠른 시작 가능

EVM 템플릿

```
cargo prove new my-project --evm
```

- Solidity 스마트 컨트랙트 포함
- 온체인 통합을 위한 완전한 구조
- Foundry 설치 필요

추가 옵션

```
# 특정 버전 사용
cargo prove new my-project --bare --version v1.2.0

# 브랜치 지정
```

```
cargo prove new my-project --evm --version main
```

생성되는 프로젝트 구조

```
my-project/
├── program/ # SP1 프로그램 소스
| ├── Cargo.toml
| └── src/
| └── main.rs
├── script/ # 증명 생성 스크립트
| ├── Cargo.toml
| ├── build.rs
| └── src/
| └── main.rs
└── contracts/ # EVM 템플릿에만 포함
├── foundry.toml
└── src/
```

12.3 프로그램 빌드 (`cargo prove build`)

기본 사용법

```
cargo prove build [OPTIONS]
```

주요 빌드 옵션

Docker 빌드

```
cargo prove build --docker
```

- 재현 가능한 빌드를 위해 Docker 컨테이너 사용
- 환경 차이로 인한 빌드 문제 해결

특정 패키지 빌드

```
cargo prove build --packages my-program
```

특정 바이너리 빌드

```
cargo prove build --binaries my-binary
```

기능 활성화

```
cargo prove build --features "feature1,feature2"
```

출력 디렉토리 지정

```
cargo prove build --output-directory ./build --elf-name my-program.elf
```

고급 빌드 옵션

Rust 플래그 추가

```
cargo prove build --rustflags "-C opt-level=3,-C target-cpu=native"
```

기본 기능 비활성화

```
cargo prove build --no-default-features
```

잠금 파일 확인

```
cargo prove build --locked
```

경고 레벨 조정

```
cargo prove build --warning-level minimal
```

12.4 툴체인 관리

툴체인 설치 (`cargo prove install-toolchain`)

```
cargo prove install-toolchain [OPTIONS]
```

GitHub 토큰 사용

```
cargo prove install-toolchain --token <GITHUB_TOKEN>
```

- API 속도 제한 우회
- 프라이빗 리포지토리 접근

툴체인 빌드 (`cargo prove build-toolchain`)

```
cargo prove build-toolchain
```

환경 변수 설정

```
# GitHub 액세스 토큰
export GITHUB_ACCESS_TOKEN=<token>

# 빌드 디렉토리 지정
export SP1_BUILD_DIR=/path/to/build

cargo prove build-toolchain
```

지원되는 플랫폼

- Linux x86_64
- Linux aarch64 (ARM64)
- macOS x86_64
- macOS aarch64 (Apple Silicon)

12.5 검증 키 관리 (`cargo prove vkey`)

ELF 파일에서 검증 키 조회

```
cargo prove vkey --elf path/to/program.elf
```

프로그램 패키지에서 검증 키 조회

```
cargo prove vkey --program my-program
```

출력 예시

```
Verification Key Hash for 'my-program':
0x1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcdef
```

12.6 프로그램 빌드 및 실행 워크플로우

1. 프로젝트 생성

```
# 새 프로젝트 생성
cargo prove new fibonacci-example --bare
cd fibonacci-example
```

2. 프로그램 작성

```
program/src/main.rs:
```

```rust
#![no_main]
sp1_zkvm::entrypoint!(main);

pub fn main() {
let n = sp1_zkvm::io::read::<u32>();
let mut a = 0u32;
let mut b = 1u32;
for _ in 0..n {
let c = a + b;
a = b;
b = c;
}
sp1_zkvm::io::commit(&a);
sp1_zkvm::io::commit(&b);
}
```

3. 스크립트 작성

script/build.rs:

```rust
fn main() {
sp1_build::build_program("../program");
}
```

script/src/main.rs:

```rust
use sp1_sdk::{include_elf, ProverClient, SP1Stdin};

const ELF: &[u8] = include_elf!("fibonacci-program");

fn main() {
let mut stdin = SP1Stdin::new();
stdin.write(&10u32);
let client = ProverClient::from_env();
let (pk, vk) = client.setup(ELF);
let proof = client.prove(&pk, &stdin).run().unwrap();
client.verify(&proof, &vk).expect("verification failed");
println!("Proof verified successfully!");
}
```

4. 빌드 및 실행

```bash
# 프로그램 빌드
cargo prove build

# 스크립트 실행 (증명 생성 및 검증)
cd script
```

```
cargo run --release
```

12.7 디버깅 및 프로파일링

실행 추적 생성

```
# 환경 변수 설정으로 상세 로그 활성화
export RUST_LOG=debug
export SP1_TRACE=1

# 프로그램 실행
cargo run --release
```

사이클 수 측정

```
// 스크립트에서 실행 통계 확인
let (_, report) = client.execute(ELF, &stdin).run().unwrap();
println!("Total cycles: {}", report.total_instruction_count());
```

메모리 사용량 모니터링

```
# 메모리 사용량 추적
export SP1_MEMORY_TRACE=1
```

```
cargo run --release
```

12.8 성능 최적화 옵션

CUDA 가속화 활성화

```
# Cargo.toml에 기능 추가
[features]
cuda = ["sp1-sdk/cuda"]

# CUDA 기능으로 빌드
cargo build --features cuda --release
```

병렬 처리 설정

```
// 환경 변수로 스레드 수 조정
std::env::set_var("RAYON_NUM_THREADS", "8");
```

메모리 최적화

```
# 샤드 크기 조정
export SP1_SHARD_SIZE=4194304 # 4MB

# 최대 사이클 수 제한
```

```
export SP1_MAX_CYCLES=1000000
```

12.9 일반적인 문제 해결

빌드 오류

툴체인 문제

```
# 툴체인 재설치
cargo prove install-toolchain

# 툴체인 확인
rustup toolchain list | grep succinct
```

의존성 문제

```
# 의존성 업데이트
cargo update

# 클린 빌드
cargo clean
cargo prove build
```

실행 오류

메모리 부족

```
# 샤드 크기 감소
export SP1_SHARD_SIZE=1048576 # 1MB

# 스왑 메모리 증가 (Linux)
sudo swapon --show
```

CUDA 오류

```
# CUDA 설치 확인
nvidia-smi

# CPU 폴백 사용
unset CUDA_VISIBLE_DEVICES
```

증명 생성 오류

검증 실패

```
// 공개 값 순서 확인
let value1 = proof.public_values.read::<u32>();
let value2 = proof.public_values.read::<u32>();
```

타임아웃

```
# 타임아웃 증가
export SP1_TIMEOUT=3600  # 1시간
```

12.10 베스트 프랙티스

개발 환경 설정

1. **버전 관리**: 특정 SP1 버전 고정
2. **환경 분리**: 개발/테스트/프로덕션 환경 구분
3. **CI/CD 통합**: 자동화된 빌드 및 테스트

성능 최적화

1. **적절한 샤드 크기**: 메모리와 성능의 균형
2. **병렬 처리 활용**: 멀티코어 시스템 최적화
3. **CUDA 가속화**: GPU 사용 가능 시 활성화

보안 고려사항

1. **검증 키 관리**: 안전한 키 저장 및 배포
2. **입력 검증**: 프로그램 입력 데이터 검증
3. **증명 무결성**: 증명 데이터 변조 방지

이 문서는 SP1 CLI 도구의 전체적인 사용법과 워크플로우를 다루며, 개발자가 효율적으로 SP1 프로젝트를 개발할 수 있도록 돕습니다.